

Python Programming

Complete Notes for Beginners to Intermediate Learners

Section	Topic
1	Introduction & Basics
2	Variables & Data Types
3	Operators
4	Control Flow
5	Loops
6	Functions
7	Data Structures (List, Tuple, Set, Dict)
8	Strings
9	Object-Oriented Programming
10	Exception Handling
11	File Handling
12	Modules & Packages
13	Useful Built-in Functions

1. Introduction & Basics

Python is a high-level, interpreted, general-purpose programming language known for its simple, readable syntax. It supports multiple programming paradigms including procedural, object-oriented, and functional programming.

Key Features

- Easy to read and write (close to plain English)
- Interpreted language (no separate compilation step)
- Dynamically typed (no need to declare variable types)
- Huge standard library and third-party ecosystem (pip)
- Cross-platform: runs on Windows, macOS, Linux

Your First Program

```
print("Hello, World!")
```

Comments

```
# This is a single-line comment

"""
This is a
multi-line comment / docstring
"""
```

2. Variables & Data Types

Variables are containers for storing data values. Python has no command for declaring a variable — it is created the moment you first assign a value to it.

```
x = 10          # int
name = "Alice"  # str
price = 9.99    # float
is_valid = True # bool
```

Common Data Types

Type	Example	Description
int	5, -12, 1000	Whole numbers
float	3.14, -0.5	Decimal numbers
str	"hello"	Text / sequence of characters
bool	True, False	Boolean values
list	[1, 2, 3]	Ordered, mutable collection

tuple	(1, 2, 3)	Ordered, immutable collection
dict	{"a": 1}	Key-value pairs
set	{1, 2, 3}	Unordered, unique values

Type Casting

```
x = int("10")      # string to int
y = float(5)       # int to float
z = str(100)       # int to string
print(type(x))     # <class 'int'>
```

3. Operators

Arithmetic Operators

```
+   Addition      a + b
-   Subtraction   a - b
*   Multiplication a * b
/   Division      a / b   (returns float)
//  Floor Division a // b  (returns int)
%   Modulus       a % b   (remainder)
**  Exponent      a ** b
```

Comparison & Logical Operators

```
==  !=  >  <  >=  <=    # Comparison
and  or  not    # Logical
```

Assignment Operators

```
x = 5
x += 2   # x = x + 2
x -= 2   # x = x - 2
x *= 2   # x = x * 2
x /= 2   # x = x / 2
```

4. Control Flow

Conditional statements let your program make decisions based on conditions.

```
age = 20

if age < 13:
    print("Child")
elif age < 20:
    print("Teenager")
else:
    print("Adult")
```

Ternary (Conditional) Expression

```
status = "Adult" if age >= 18 else "Minor"
```

5. Loops

For Loop

```
for i in range(5):  
    print(i)          # 0 1 2 3 4  
  
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

While Loop

```
count = 0  
while count < 5:  
    print(count)  
    count += 1
```

Loop Control Statements

```
break      # exits the loop entirely  
continue  # skips to the next iteration  
pass      # does nothing (placeholder)
```

6. Functions

A function is a reusable block of code that performs a specific task.

```
def greet(name, greeting="Hello"):
    """Returns a greeting message."""
    return f"{greeting}, {name}!"

print(greet("Alice"))          # Hello, Alice!
print(greet("Bob", "Hi"))      # Hi, Bob!
```

Lambda (Anonymous) Functions

```
square = lambda x: x ** 2
print(square(5))    # 25
```

*args and **kwargs

```
def demo(*args, **kwargs):
    print(args)      # tuple of positional args
    print(kwargs)    # dict of keyword args

demo(1, 2, name="Alice")
```

7. Data Structures

Lists (ordered, mutable)

```
nums = [1, 2, 3]
nums.append(4)      # [1, 2, 3, 4]
nums.remove(2)      # [1, 3, 4]
nums[0] = 10        # [10, 3, 4]
print(len(nums))    # 3
print(nums[-1])     # last element
print(nums[0:2])    # slicing -> [10, 3]
```

Tuples (ordered, immutable)

```
point = (3, 4)
x, y = point        # unpacking
print(point[0])     # 3
```

Dictionaries (key-value pairs)

```
person = {"name": "Alice", "age": 25}
print(person["name"]) # Alice
person["city"] = "Delhi" # add new key
for key, value in person.items():
    print(key, value)
```

Sets (unordered, unique values)

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)    # union -> {1,2,3,4}
print(a & b)    # intersection -> {2,3}
print(a - b)    # difference -> {1}
```

Comprehensions

```
squares = [x**2 for x in range(6)]
evens = [x for x in range(10) if x % 2 == 0]
square_dict = {x: x**2 for x in range(4)}
```

8. Strings

```
s = "Hello, Python!"
print(s.upper())      # HELLO, PYTHON!
print(s.lower())      # hello, python!
print(s.replace("Hello", "Hi"))
print(s.split(","))   # ['Hello', ' Python!']
print(len(s))         # length
print(s[0:5])         # slicing -> Hello
```

F-strings (formatted string literals)

```
name = "Alice"
age = 25
print(f"{name} is {age} years old")
```

9. Object-Oriented Programming

Classes are blueprints for creating objects that bundle data (attributes) and behavior (methods) together.

```
class Dog:
    species = "Canine"          # class attribute

    def __init__(self, name, age):
        self.name = name        # instance attribute
        self.age = age

    def bark(self):
        return f"{self.name} says Woof!"

d = Dog("Rex", 3)
print(d.bark())                # Rex says Woof!
```

Inheritance

```
class Animal:
    def speak(self):
        return "Some sound"

class Cat(Animal):
    def speak(self):
        return "Meow"

c = Cat()
print(c.speak())               # Meow
```

Key OOP Concepts

- Encapsulation — bundling data and methods within a class
- Inheritance — a class deriving properties from another class
- Polymorphism — same method name behaving differently across classes
- Abstraction — hiding complex implementation details

10. Exception Handling

Exceptions handle runtime errors gracefully so the program doesn't crash.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
except Exception as e:
    print(f"Error: {e}")
else:
    print("No errors occurred")
finally:
```

```
print("This always runs")
```

Raising Custom Exceptions

```
if age < 0:  
    raise ValueError("Age cannot be negative")
```

11. File Handling

```
# Writing to a file  
with open("notes.txt", "w") as f:  
    f.write("Hello, file!")  
  
# Reading from a file  
with open("notes.txt", "r") as f:  
    content = f.read()  
    print(content)
```

The 'with' statement automatically closes the file, even if an error occurs — always prefer it over manually calling `open()/close()`.

12. Modules & Packages

A module is simply a .py file containing functions/classes that can be imported into another file. A package is a folder of related modules.

```
import math
print(math.sqrt(16))      # 4.0

from datetime import datetime
print(datetime.now())

import numpy as np        # third-party package (pip install numpy)
```

13. Useful Built-in Functions

Function	Purpose
len(x)	Number of items in a sequence
type(x)	Type of an object
range(a, b)	Sequence of numbers from a to b-1
sorted(x)	Returns a new sorted list
enumerate(x)	Index + value pairs while looping
zip(a, b)	Combines two iterables element-wise
map(func, x)	Applies a function to every item
filter(func, x)	Filters items based on a condition
input()	Reads user input as a string

End of Notes — Keep practicing by writing small programs, and refer back to these notes whenever you need a quick refresher!